

操作 Docker 容器

容器是 Docker 又一核心概念。

简单的说，容器是独立运行的一个或一组应用，以及它们的运行态环境。对应的，虚拟机可以理解为模拟运行的一整套操作系统（提供了运行态环境和其他系统环境）和跑在上面的应用。

本章将具体介绍如何来管理一个容器，包括创建、启动和停止等。

启动容器

启动容器有两种方式，一种是基于镜像新建一个容器并启动，另外一个是在终止状态（`stopped`）的容器重新启动。

因为 **Docker** 的容器实在太轻量级了，很多时候用户都是随时删除和新创建容器。

新建并启动

所需要的命令主要为 `docker run`。

例如，下面的命令输出一个“Hello World”，之后终止容器。

```
$ docker run ubuntu:14.04 /bin/echo 'Hello world'
Hello world
```

这跟在本地直接执行 `/bin/echo 'hello world'` 几乎感觉不出任何区别。

下面的命令则启动一个 **bash** 终端，允许用户进行交互。

```
$ docker run -t -i ubuntu:14.04 /bin/bash
root@af8bae53bdd3:/#
```

其中，`-t` 选项让 **Docker** 分配一个伪终端（**pseudo-tty**）并绑定到容器的标准输入上，`-i` 则让容器的标准输入保持打开。

在交互模式下，用户可以通过所创建的终端来输入命令，例如

```
root@af8bae53bdd3:/# pwd
/
root@af8bae53bdd3:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
```

当利用 `docker run` 来创建容器时，**Docker** 在后台运行的标准操作包括：

- 检查本地是否存在指定的镜像，不存在就从公有仓库下载
- 利用镜像创建并启动一个容器
- 分配一个文件系统，并在只读的镜像层外面挂载一层可读写层
- 从宿主主机配置的网桥接口中桥接一个虚拟接口到容器中去
- 从地址池配置一个 **ip** 地址给容器
- 执行用户指定的应用程序
- 执行完毕后容器被终止

启动已终止容器

可以利用 `docker container start` 命令，直接将一个已经终止的容器启动运行。

容器的核心为所执行的应用程序，所需要的资源都是应用程序运行所必需的。除此之外，并没有其它的资源。可以在伪终端中利用 `ps` 或 `top` 来查看进程信息。

```
root@ba267838cc1b:/# ps
PID TTY          TIME CMD
  1 ?            00:00:00 bash
 11 ?            00:00:00 ps
```

可见，容器中仅运行了指定的 `bash` 应用。这种特点使得 `Docker` 对资源的利用率极高，是货真价实的轻量级虚拟化。

后台运行

更多的时候，需要让 **Docker** 在后台运行而不是直接把执行命令的结果输出在当前宿主机下。此时，可以通过添加 `-d` 参数来实现。

下面举两个例子来说明一下。

如果不使用 `-d` 参数运行容器。

```
$ docker run ubuntu:17.10 /bin/sh -c "while true; do echo hello world; sleep 1; done"
hello world
hello world
hello world
hello world
```

容器会把输出的结果 (STDOUT) 打印到宿主机上面

如果使用了 `-d` 参数运行容器。

```
$ docker run -d ubuntu:17.10 /bin/sh -c "while true; do echo hello world; sleep 1; done"
77b2dc01fe0f3f1265df143181e7b9af5e05279a884f4776ee75350ea9d8017a
```

此时容器会在后台运行并不会把输出的结果 (STDOUT) 打印到宿主机上面(输出结果可以用 `docker logs` 查看)。

注：容器是否会长久运行，是和 `docker run` 指定的命令有关，和 `-d` 参数无关。

使用 `-d` 参数启动后会返回一个唯一的 id，也可以通过 `docker container ls` 命令来查看容器信息。

```
$ docker container ls
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
77b2dc01fe0f   ubuntu:17.10  /bin/sh -c 'while tr    2 minutes ago Up 1 minute          agitated_wright
```

要获取容器的输出信息，可以通过 `docker container logs` 命令。

```
$ docker container logs [container ID or NAMES]
hello world
hello world
hello world
. . .
```

终止容器

可以使用 `docker container stop` 来终止一个运行中的容器。

此外，当 Docker 容器中指定的应用终结时，容器也自动终止。

例如对于上一章节中只启动了一个终端的容器，用户通过 `exit` 命令或 `Ctrl+d` 来退出终端时，所创建的容器立刻终止。

终止状态的容器可以用 `docker container ls -a` 命令看到。例如

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
ba267838cc1b	ubuntu:14.04	"/bin/bash"	30 minutes ago
Exited (0) About a minute ago		trusting_newton	
98e5efa7d997	training/webapp:latest	"python app.py"	About an hour ago
Exited (0) 34 minutes ago		backstabbing_pike	

处于终止状态的容器，可以通过 `docker container start` 命令来重新启动。

此外，`docker container restart` 命令会将一个运行态的容器终止，然后再重新启动它。

进入容器

在使用 `-d` 参数时，容器启动后会进入后台。

某些时候需要进入容器进行操作，包括使用 `docker attach` 命令或 `docker exec` 命令，推荐大家使用 `docker exec` 命令，原因会在下面说明。

attach 命令

`docker attach` 是 Docker 自带的命令。下面示例如何使用该命令。

```
$ docker run -dit ubuntu
243c32535da7d142fb0e6df616a3c3ada0b8ab417937c853a9e1c251f499f550

$ docker container ls
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
243c32535da7       ubuntu:latest      "/bin/bash"        18 seconds ago     Up 17 seconds
nostalgic_hypatia

$ docker attach 243c
root@243c32535da7:/#
```

注意：如果从这个 `stdin` 中 `exit`，会导致容器的停止。

exec 命令

-i -t 参数

`docker exec` 后边可以跟多个参数，这里主要说明 `-i` `-t` 参数。

只用 `-i` 参数时，由于没有分配伪终端，界面没有我们熟悉的 Linux 命令提示符，但命令执行结果仍然可以返回。

当 `-i` `-t` 参数一起使用时，则可以看到我们熟悉的 Linux 命令提示符。

```
$ docker run -dit ubuntu
69d137adef7a8a689cbcb059e94da5489d3cddd240ff675c640c8d96e84fe1f6

$ docker container ls
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
69d137adef7a        ubuntu:latest      "/bin/bash"        18 seconds ago     Up 17
seconds            zealous_swirles

$ docker exec -i 69d1 bash
ls
bin
boot
dev
...

$ docker exec -it 69d1 bash
root@69d137adef7a:/#
```

如果从这个 `stdin` 中 `exit`，不会导致容器的停止。这就是为什么推荐大家使用 `docker exec` 的原因。

更多参数说明请使用 `docker exec --help` 查看。

导出和导入容器

导出容器

如果要导出本地某个容器，可以使用 `docker export` 命令。

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
	PORTS	NAMES		
7691a814370e	ubuntu:14.04	"/bin/bash"	36 hours ago	Exited
(0) 21 hours ago		test		

```
$ docker export 7691a814370e > ubuntu.tar
```

这样将导出容器快照到本地文件。

导入容器快照

可以使用 `docker import` 从容器快照文件中再导入为镜像，例如

```
$ cat ubuntu.tar | docker import - test/ubuntu:v1.0
```

```
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	VIRTUAL SIZE
test/ubuntu	v1.0	9d37a6082e97	About a minute ago	171.3 MB

此外，也可以通过指定 URL 或者某个目录来导入，例如

```
$ docker import http://example.com/exampleimage.tgz example/imagerepo
```

注：用户既可以使用 `docker load` 来导入镜像存储文件到本地镜像库，也可以使用 `docker import` 来导入一个容器快照到本地镜像库。这两者的区别在于容器快照文件将丢弃所有的历史记录和元数据信息（即仅保存容器当时的快照状态），而镜像存储文件将保存完整记录，体积也要大。此外，从容器快照文件导入时可以重新指定标签等元数据信息。

删除容器

可以使用 `docker container rm` 来删除一个处于终止状态的容器。例如

```
$ docker container rm trusting_newton
trusting_newton
```

如果要删除一个运行中的容器，可以添加 `-f` 参数。Docker 会发送 `SIGKILL` 信号给容器。

清理所有处于终止状态的容器

用 `docker container ls -a` 命令可以查看所有已经创建的包括终止状态的容器，如果数量太多要一个个删除可能会很麻烦，用下面的命令可以清理掉所有处于终止状态的容器。

```
$ docker container prune
```